22 Input/Output

In man-machine symbiosis, it is man who must adjust: The machines can't.

C's input/output library is the biggest and most important part of the standard library. As befits its lofty status, we'll devote an entire chapter (the longest in the book) to the `<stdio.h>` header, the primary repository of input/output functions.

We've been using `<stdio.h>` since Chapter 2, and we have experience with the `printf`, `scanf`, `putchar`, `getchar`, `puts`, and `gets` functions. This chapter provides more information about these six functions, as well as introducing a host of new functions, most of which deal with files. Fortunately, many of the new functions are closely related to functions with which we're already acquainted. `fprintf`, for instance, is the "file version" of the `printf` function.

We'll start the chapter with a discussion of some basic issues: the stream concept, the `FILE` type, input and output redirection, and the difference between text files and binary files (Section 22.1). We'll then turn to functions that are designed specifically for use with files, including functions that open and close files (Section 22.2). After covering `printf`, `scanf`, and related functions for "formatted" input/output (Section 22.3), we'll look at functions that read and write unformatted data:

- `getc`, `putc`, and related functions, which read and write one *character* at a time (Section 22.4).
- `gets`, `puts`, and related functions, which read and write one *line* at a time (Section 22.5).
- `fread` and `fwrite`, which read and write *blocks* of data (Section 22.6).

Section 22.7 then shows how to perform random access operations on files. Finally, Section 22.8 describes the `sprintf`, `snprintf`, and `sscanf` functions, variants of `printf` and `scanf` that write to a string or read from a string.

This chapter covers all but eight of the functions in `<stdio.h>`. One of these eight, the `perror` function, is closely related to the `<errno.h>` header, so

I'll postpone it until Section 24.2, which discusses that header. Section 26.1 covers the remaining functions (`vfprintf`, `vprintf`, `vsprintf`, `vsnprintf`, `vfscanf`, `vscanf`, and `vsscanf`). These functions rely on the `va_list` type, which is introduced in that section.

C99

<wchar.h> header ➤25.5

In C89, all standard input/output functions belong to `<stdio.h>`, but such is not the case in C99, where some I/O functions are declared in the `<wchar.h>` header. The `<wchar.h>` functions deal with wide characters rather than ordinary characters; the good news is that most of these functions closely resemble those of `<stdio.h>`. Functions in `<stdio.h>` that read or write data are known as ***byte input/output functions;*** similar functions in `<wchar.h>` are called ***wide-character input/output functions.***

22.1 Streams

In C, the term ***stream*** means any source of input or any destination for output. Many small programs, like the ones in previous chapters, obtain all their input from one stream (usually associated with the keyboard) and write all their output to another stream (usually associated with the screen).

Larger programs may need additional streams. These streams often represent files stored on various media (such as hard drives, CDs, DVDs, and flash memory), but they could just as easily be associated with devices that don't store files: network ports, printers, and the like. We'll concentrate on files, since they're common and easy to understand. (I may even occasionally use the term *file* when I should say *stream*.) Keep in mind, however, that many of the functions in `<stdio.h>` work equally well with all streams, not just the ones that represent files.

File Pointers

Accessing a stream in a C program is done through a ***file pointer***, which has type `FILE *` (the `FILE` type is declared in `<stdio.h>`). Certain streams are represented by file pointers with standard names; we can declare additional file pointers as needed. For example, if a program needs two streams in addition to the standard ones, it might contain the following declaration:

```
FILE *fp1, *fp2;
```

A program may declare any number of `FILE *` variables, although operating systems usually limit the number of streams that can be open at one time.

Standard Streams and Redirection

`<stdio.h>` provides three standard streams (Table 22.1). These streams are ready to use—we don't declare them, and we don't open or close them.

Table 22.1
Standard Streams

File Pointer	*Stream*	*Default Meaning*
`stdin`	Standard input	Keyboard
`stdout`	Standard output	Screen
`stderr`	Standard error	Screen

The functions that we've used in previous chapters—`printf`, `scanf`, `putchar`, `getchar`, `puts`, and `gets`—obtain input from `stdin` and send output to `stdout`. By default, `stdin` represents the keyboard; `stdout` and `stderr` represent the screen. However, many operating systems allow these default meanings to be changed via a mechanism known as ***redirection***.

Q&A

Typically, we can force a program to obtain its input from a file instead of from the keyboard by putting the name of the file on the command line, preceded by the < character:

```
demo <in.dat
```

This technique, known as ***input redirection***, essentially makes the `stdin` stream represent a file (`in.dat`, in this case) instead of the keyboard. The beauty of redirection is that the `demo` program doesn't realize that it's reading from `in.dat`; as far as it knows, any data it obtains from `stdin` is being entered at the keyboard.

Output redirection is similar. Redirecting the `stdout` stream is usually done by putting a file name on the command line, preceded by the > character:

```
demo >out.dat
```

All data written to `stdout` will now go into the `out.dat` file instead of appearing on the screen. Incidentally, we can combine output redirection with input redirection:

```
demo <in.dat >out.dat
```

The < and > characters don't have to be adjacent to file names, and the order in which the redirected files are listed doesn't matter, so the following examples would work just as well:

```
demo < in.dat > out.dat
demo >out.dat <in.dat
```

One problem with output redirection is that *everything* written to `stdout` is put into a file. If the program goes off the rails and begins writing error messages, we won't see them until we look at the file. This is where `stderr` comes in. By writing error messages to `stderr` instead of `stdout`, we can guarantee that those messages will appear on the screen even when `stdout` has been redirected. (Operating systems often allow `stderr` itself to be redirected, though.)

Text Files versus Binary Files

`<stdio.h>` supports two kinds of files: text and binary. The bytes in a ***text file*** represent characters, making it possible for a human to examine the file or edit it.

The source code for a C program is stored in a text file, for example. In a ***binary file***, on the other hand, bytes don't necessarily represent characters; groups of bytes might represent other types of data, such as integers and floating-point numbers. An executable C program is stored in a binary file, as you'll quickly realize if you try to look at the contents of one.

Text files have two characteristics that binary files don't possess:

- ***Text files are divided into lines.*** Each line in a text file normally ends with one or two special characters; the choice of characters depends on the operating system. In Windows, the end-of-line marker is a carriage-return character (`'\x0d'`) followed immediately by a line-feed character (`'\x0a'`). In UNIX and newer versions of the Macintosh operating system (Mac OS), the end-of-line marker is a single line-feed character. Older versions of Mac OS use a single carriage-return character.

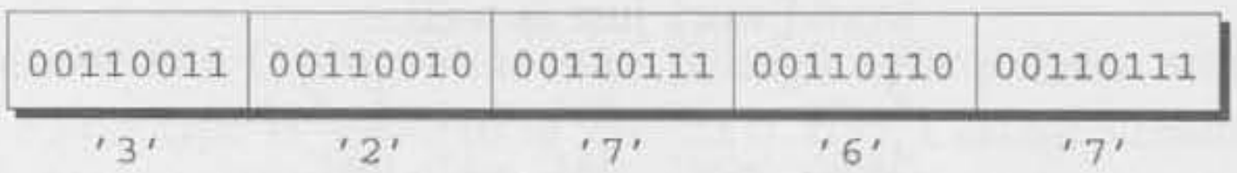
Q&A

- ***Text files may contain a special "end-of-file" marker.*** Some operating systems allow a special byte to be used as a marker at the end of a text file. In Windows, the marker is `'\x1a'` (Ctrl-Z). There's no requirement that Ctrl-Z be present, but if it is, it marks the end of the file; any bytes after Ctrl-Z are to be ignored. The Ctrl-Z convention is a holdover from DOS, which in turn inherited it from CP/M, an early operating system for personal computers. Most other operating systems, including UNIX, have no special end-of-file character.

Binary files aren't divided into lines. In a binary file, there are no end-of-line or end-of-file markers; all bytes are treated equally.

Q&A

When we write data to a file, we'll need to consider whether to store it in text form or in binary form. To see the difference, consider how we might store the number 32767 in a file. One option would be to write the number in text form as the characters `3`, `2`, `7`, `6`, and `7`. If the character set is ASCII, we'd have the following five bytes:



The other option is to store the number in binary, which would take as few as two bytes:



little-endian order ➤ *20.3*

(The bytes will be reversed on systems that store data in little-endian order.) As this example shows, storing numbers in binary can often save quite a bit of space.

When we're writing a program that reads from a file or writes to a file, we need to take into account whether it's a text file or a binary file. A program that displays the contents of a file on the screen will probably assume it's a text file. A file-

copying program, on the other hand, can't assume that the file to be copied is a text file. If it does, binary files containing an end-of-file character won't be copied completely. When we can't say for sure whether a file is text or binary, it's safer to assume that it's binary.

22.2 File Operations

Simplicity is one of the attractions of input and output redirection; there's no need to open a file, close a file, or perform any other explicit file operations. Unfortunately, redirection is too limited for many applications. When a program relies on redirection, it has no control over its files; it doesn't even know their names. Worse still, redirection doesn't help if the program needs to read from two files or write to two files at the same time.

When redirection isn't enough, we'll end up using the file operations that `<stdio.h>` provides. In this section, we'll explore these operations, which include opening a file, closing a file, changing the way a file is buffered, deleting a file, and renaming a file.

Opening a File

```
FILE *fopen(const char * restrict filename,
            const char * restrict mode);
```

fopen

Opening a file for use as a stream requires a call of the `fopen` function. `fopen`'s first argument is a string containing the name of the file to be opened. (A "file name" may include information about the file's location, such as a drive specifier or path.) The second argument is a "mode string" that specifies what operations we intend to perform on the file. The string `"r"`, for instance, indicates that data will be read from the file, but none will be written to it.

restrict keyword ➤17.8

C99

Note that `restrict` appears twice in the prototype for the `fopen` function. `restrict`, which is a C99 keyword, indicates that `filename` and `mode` should point to strings that don't share memory locations. The C89 prototype for `fopen` doesn't contain `restrict` but is otherwise identical. `restrict` has no effect on the behavior of `fopen`, so it can usually just be ignored. In this and subsequent chapters, I'll italicize `restrict` as a reminder that it's a C99 feature.



Windows programmers: Be careful when the file name in a call of `fopen` includes the \ character, since C treats \ as the beginning of an escape sequence. The call

escape sequences ➤7.3

```
fopen("c:\project\test1.dat", "r")
```

will fail, because the compiler treats `\t` as a character escape. (`\p` isn't a valid character escape, but it looks like one. The C standard states that its meaning is

undefined.) There are two ways to avoid the problem. One is to use \\ instead of \:

```
fopen("c:\\project\\test1.dat", "r")
```

The other technique is even easier—just use the / character instead of \:

```
fopen("c:/project/test1.dat", "r")
```

Windows will happily accept / instead of \ as the directory separator.

`fopen` returns a file pointer that the program can (and usually will) save in a variable and use later whenever it needs to perform an operation on the file. Here's a typical call of `fopen`, where `fp` is a variable of type `FILE *`:

```
fp = fopen("in.dat", "r");   /* opens in.dat for reading */
```

When the program calls an input function to read from `in.dat` later, it will supply `fp` as an argument.

When it can't open a file, `fopen` returns a null pointer. Perhaps the file doesn't exist, or it's in the wrong place, or we don't have permission to open it.



Never assume that a file can be opened; always test the return value of `fopen` to make sure it's not a null pointer.

Modes

Which mode string we'll pass to `fopen` depends not only on what operations we plan to perform on the file later but also on whether the file contains text or binary data. To open a text file, we'd use one of the mode strings in Table 22.2.

Table 22.2 Mode Strings for Text Files

String	*Meaning*
`"r"`	Open for reading
`"w"`	Open for writing (file need not exist)
`"a"`	Open for appending (file need not exist)
`"r+"`	Open for reading and writing, starting at beginning
`"w+"`	Open for reading and writing (truncate if file exists)
`"a+"`	Open for reading and writing (append if file exists)

Q&A

When we use `fopen` to open a binary file, we'll need to include the letter `b` in the mode string. Table 22.3 lists mode strings for binary files.

From Tables 22.2 and 22.3, we see that `<stdio.h>` distinguishes between *writing* data and *appending* data. When data is written to a file, it normally overwrites what was previously there. When a file is opened for appending, however, data written to the file is added at the end, thus preserving the file's original contents.

By the way, special rules apply when a file is opened for both reading and writing (the mode string contains the + character). We can't switch from reading to writ-

Table 22.3 Mode Strings for Binary Files

String	*Meaning*
`"rb"`	Open for reading
`"wb"`	Open for writing (file need not exist)
`"ab"`	Open for appending (file need not exist)
`"r+b"` or `"rb+"`	Open for reading and writing, starting at beginning
`"w+b"` or `"wb+"`	Open for reading and writing (truncate if file exists)
`"a+b"` or `"ab+"`	Open for reading and writing (append if file exists)

file-positioning functions ➤22.7

ing without first calling a file-positioning function unless the reading operation encountered the end of the file. Also, we can't switch from writing to reading without either calling `fflush` (covered later in this section) or calling a file-positioning function.

Closing a File

```
int fclose(FILE *stream);
```

fclose

The `fclose` function allows a program to close a file that it's no longer using. The argument to `fclose` must be a file pointer obtained from a call of `fopen` or `freopen` (discussed later in this section). `fclose` returns zero if the file was closed successfully; otherwise, it returns the error code `EOF` (a macro defined in `<stdio.h>`).

Q&A

To show how `fopen` and `fclose` are used in practice, here's the outline of a program that opens the file `example.dat` for reading, checks that it was opened successfully, then closes it before terminating:

```
#include <stdio.h>
#include <stdlib.h>

#define FILE_NAME "example.dat"

int main(void)
{
  FILE *fp;

  fp = fopen(FILE_NAME, "r");
  if (fp == NULL) {
    printf("Can't open %s\n", FILE_NAME);
    exit(EXIT_FAILURE);
  }
  ...
  fclose(fp);
  return 0;
}
```

Of course, C programmers being the way they are, it's not unusual to see the call of `fopen` combined with the declaration of `fp`:

```
FILE *fp = fopen(FILE_NAME, "r");
```

or the test against NULL:

```
if ((fp = fopen(FILE_NAME, "r")) == NULL) …
```

Attaching a File to an Open Stream

```
FILE *freopen(const char * restrict filename,
              const char * restrict mode,
              FILE * restrict stream);
```

freopen

`freopen` attaches a different file to a stream that's already open. The most common use of `freopen` is to associate a file with one of the standard streams (`stdin`, `stdout`, or `stderr`). To cause a program to begin writing to the file `foo`, for instance, we could use the following call of `freopen`:

```
if (freopen("foo", "w", stdout) == NULL) {
  /* error; foo can't be opened */
}
```

After closing any file previously associated with `stdout` (by command-line redirection or a previous call of `freopen`), `freopen` will open `foo` and associate it with `stdout`.

`freopen`'s normal return value is its third argument (a file pointer). If it can't open the new file, `freopen` returns a null pointer. (`freopen` ignores the error if the old file can't be closed.)

C99

C99 adds a new twist. If `filename` is a null pointer, `freopen` attempts to change the stream's mode to that specified by the `mode` parameter. Implementations aren't required to support this feature, however; if they do, they may place restrictions on which mode changes are permitted.

Obtaining File Names from the Command Line

When we're writing a program that will need to open a file, one problem soon becomes apparent: how do we supply the file name to the program? Building file names into the program itself doesn't provide much flexibility, and prompting the user to enter file names can be awkward. Often, the best solution is to have the program obtain file names from the command line. When we execute a program named `demo`, for example, we might supply it with file names by putting them on the command line:

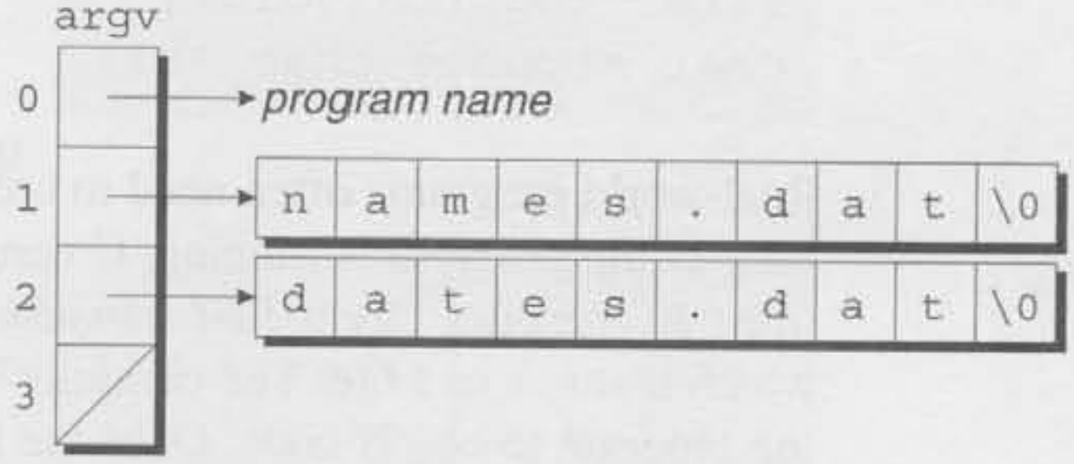
Q&A

```
demo names.dat dates.dat
```

In Section 13.7, we saw how to access command-line arguments by defining `main` as a function with two parameters:

```
int main(int argc, char *argv[])
{
  …
}
```

`argc` is the number of command-line arguments; `argv` is an array of pointers to the argument strings. `argv[0]` points to the program name, `argv[1]` through `argv[argc-1]` point to the remaining arguments, and `argv[argc]` is a null pointer. In the example above, `argc` is 3, `argv[0]` points to a string containing the program name, `argv[1]` points to the string `"names.dat"`, and `argv[2]` points to the string `"dates.dat"`:



PROGRAM

Checking Whether a File Can Be Opened

The following program determines if a file exists and can be opened for reading. When the program is run, the user will give it a file name to check:

`canopen` *file*

The program will then print either *file* `can be opened` or *file* `can't be opened`. If the user enters the wrong number of arguments on the command line, the program will print the message `usage: canopen filename` to remind the user that `canopen` requires a single file name.

canopen.c

```
/* Checks whether a file can be opened for reading */

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
  FILE *fp;

  if (argc != 2) {
    printf("usage: canopen filename\n");
    exit(EXIT_FAILURE);
  }

  if ((fp = fopen(argv[1], "r")) == NULL) {
    printf("%s can't be opened\n", argv[1]);
    exit(EXIT_FAILURE);
  }

  printf("%s can be opened\n", argv[1]);
  fclose(fp);
  return 0;
}
```

Note that we can use redirection to discard the output of `canopen` and simply test the status value it returns.

Temporary Files

```
FILE *tmpfile(void);
char *tmpnam(char *s);
```

Real-world programs often need to create temporary files—files that exist only as long as the program is running. C compilers, for instance, often create temporary files. A compiler might first translate a C program to some intermediate form, which it stores in a file. The compiler would then read the file later as it translates the program to object code. Once the program is completely compiled, there's no need to preserve the file containing the program's intermediate form. `<stdio.h>` provides two functions, `tmpfile` and `tmpnam`, for working with temporary files.

tmpfile

`tmpfile` creates a temporary file (opened in `"wb+"` mode) that will exist until it's closed or the program ends. A call of `tmpfile` returns a file pointer that can be used to access the file later:

```
FILE *tempptr;
...
tempptr = tmpfile();    /* creates a temporary file */
```

If it fails to create a file, `tmpfile` returns a null pointer.

Although `tmpfile` is easy to use, it has a couple of drawbacks: (1) we don't know the name of the file that `tmpfile` creates, and (2) we can't decide later to make the file permanent. If these restrictions turn out to be a problem, the alternative is to create a temporary file using `fopen`. Of course, we don't want this file to have the same name as a previously existing file, so we need some way to generate new file names; that's where the `tmpnam` function comes in.

tmpnam

`tmpnam` generates a name for a temporary file. If its argument is a null pointer, `tmpnam` stores the file name in a static variable and returns a pointer to it:

```
char *filename;
...
filename = tmpnam(NULL);    /* creates a temporary file name */
```

Otherwise, `tmpnam` copies the file name into a character array provided by the programmer:

```
char filename[L_tmpnam];
...
tmpnam(filename);           /* creates a temporary file name */
```

In the latter case, `tmpnam` also returns a pointer to the first character of this array. `L_tmpnam` is a macro in `<stdio.h>` that specifies how long to make a character array that will hold a temporary file name.



Be sure that `tmpnam`'s argument points to an array of at least `L_tmpnam` characters. Also, be careful not to call `tmpnam` too often; the `TMP_MAX` macro (defined in `<stdio.h>`) specifies the maximum number of temporary file names that can potentially be generated by `tmpnam` during the execution of a program. If it fails to generate a file name, `tmpnam` returns a null pointer.

File Buffering

```
int fflush(FILE *stream);
void setbuf(FILE * restrict stream,
            char * restrict buf);
int setvbuf(FILE * restrict stream,
            char * restrict buf,
            int mode, size_t size);
```

Transferring data to or from a disk drive is a relatively slow operation. As a result, it isn't feasible for a program to access a disk file directly each time it wants to read or write a byte. The secret to achieving acceptable performance is ***buffering:*** data written to a stream is actually stored in a buffer area in memory; when it's full (or the stream is closed), the buffer is "flushed" (written to the actual output device). Input streams can be buffered in a similar way: the buffer contains data from the input device; input is read from this buffer instead of the device itself. Buffering can result in enormous gains in efficiency, since reading a byte from a buffer or storing a byte in a buffer takes hardly any time at all. Of course, it takes time to transfer the buffer contents to or from disk, but one large "block move" is much faster than many tiny byte moves.

The functions in `<stdio.h>` perform buffering automatically when it seems advantageous. The buffering takes place behind the scenes, and we usually don't worry about it. On rare occasions, though, we may need to take a more active role. If so, we can use the functions `fflush`, `setbuf`, and `setvbuf`.

fflush

When a program writes output to a file, the data normally goes into a buffer first. The buffer is flushed automatically when it's full or the file is closed. By calling `fflush`, however, a program can flush a file's buffer as often as it wishes. The call

Q&A

```
fflush(fp);      /* flushes buffer for fp */
```

flushes the buffer for the file associated with `fp`. The call

```
fflush(NULL);    /* flushes all buffers */
```

flushes *all* output streams. `fflush` returns zero if it's successful and `EOF` if an error occurs.

setvbuf

`setvbuf` allows us to change the way a stream is buffered and to control the size and location of the buffer. The function's third argument, which specifies the kind of buffering desired, should be one of the following macros:

- `_IOFBF` (full buffering). Data is read from the stream when the buffer is empty or written to the stream when it's full.
- `_IOLBF` (line buffering). Data is read from the stream or written to the stream one line at a time.
- `_IONBF` (no buffering). Data is read from the stream or written to the stream directly, without a buffer.

(All three macros are defined in `<stdio.h>`.) Full buffering is the default for streams that aren't connected to interactive devices.

`setvbuf`'s second argument (if it's not a null pointer) is the address of the desired buffer. The buffer might have static storage duration, automatic storage duration, or even be allocated dynamically. Making the buffer automatic allows its space to be reclaimed automatically at block exit; allocating it dynamically enables us to free the buffer when it's no longer needed. `setvbuf`'s last argument is the number of bytes in the buffer. A larger buffer may give better performance; a smaller buffer saves space.

For example, the following call of `setvbuf` changes the buffering of `stream` to full buffering, using the `N` bytes in the `buffer` array as the buffer:

```
char buffer[N];
...
setvbuf(stream, buffer, _IOFBF, N);
```



`setvbuf` must be called after `stream` is opened but before any other operations are performed on it.

It's also legal to call `setvbuf` with a null pointer as the second argument, which requests that `setvbuf` create a buffer with the specified size. `setvbuf` returns zero if it's successful. It returns a nonzero value if the `mode` argument is invalid or the request can't be honored.

setbuf

`setbuf` is an older function that assumes default values for the buffering mode and buffer size. If `buf` is a null pointer, the call `setbuf(stream, buf)` is equivalent to

```
(void) setvbuf(stream, NULL, _IONBF, 0);
```

Otherwise, it's equivalent to

```
(void) setvbuf(stream, buf, _IOFBF, BUFSIZ);
```

where `BUFSIZ` is a macro defined in `<stdio.h>`. The `setbuf` function is considered obsolete; it's not recommended for use in new programs.



When using `setvbuf` or `setbuf`, be sure to close the stream before its buffer is deallocated. In particular, if the buffer is local to a function and has automatic storage duration, be sure to close the stream before the function returns.

Miscellaneous File Operations

```
int remove(const char *filename);
int rename(const char *old, const char *new);
```

The functions `remove` and `rename` allow a program to perform basic file management operations. Unlike most other functions in this section, `remove` and `rename` work with file *names* instead of file *pointers*. Both functions return zero if they succeed and a nonzero value if they fail.

remove

`remove` deletes a file:

```
remove("foo");              /* deletes the file named "foo" */
```

If a program uses `fopen` (instead of `tmpfile`) to create a temporary file, it can use `remove` to delete the file before the program terminates. Be sure that the file to be removed has been closed; the effect of removing a file that's currently open is implementation-defined.

rename

`rename` changes the name of a file:

```
rename("foo", "bar");    /* renames "foo" to "bar" */
```

`rename` is handy for renaming a temporary file created using `fopen` if a program should decide to make it permanent. If a file with the new name already exists, the effect is implementation-defined.



If the file to be renamed is open, be sure to close it before calling `rename`; the function may fail if asked to rename an open file.

22.3 Formatted I/O

In this section, we'll examine library functions that use format strings to control reading and writing. These functions, which include our old friends `printf` and `scanf`, have the ability to convert data from character form to numeric form during input and from numeric form to character form during output. None of the other I/O functions can do such conversions.

The ...printf Functions

```
int fprintf(FILE * restrict stream,
            const char * restrict format, ...);
int printf(const char * restrict format, ...);
```

fprintf
printf

The `fprintf` and `printf` functions write a variable number of data items to an output stream, using a format string to control the appearance of the output.

ellipsis ➤26.1

The prototypes for both functions end with the `...` symbol (an ***ellipsis***), which indicates a variable number of additional arguments. Both functions return the number of characters written; a negative return value indicates that an error occurred.

The only difference between `printf` and `fprintf` is that `printf` always writes to `stdout` (the standard output stream), whereas `fprintf` writes to the stream indicated by its first argument:

```
printf("Total: %d\n", total);          /* writes to stdout */
fprintf(fp, "Total: %d\n", total);     /* writes to fp */
```

A call of `printf` is equivalent to a call of `fprintf` with `stdout` as the first argument.

Don't think of `fprintf` as merely a function that writes data to disk files, though. Like many functions in `<stdio.h>`, `fprintf` works fine with any output stream. In fact, one of the most common uses of `fprintf`—writing error messages to `stderr`, the standard error stream—has nothing to do with disk files. Here's what such a call might look like:

```
fprintf(stderr, "Error: data file can't be opened.\n");
```

Writing the message to `stderr` guarantees that it will appear on the screen even if the user redirects `stdout`.

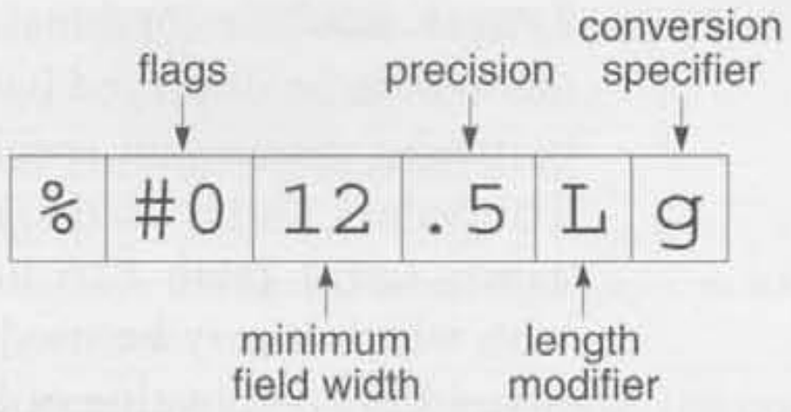
v...printf functions ➤26.1

There are two other functions in `<stdio.h>` that can write formatted output to a stream. These functions, named `vfprintf` and `vprintf`, are fairly obscure. Both rely on the `va_list` type, which is declared in `<stdarg.h>`, so they're discussed along with that header.

...printf Conversion Specifications

Both `printf` and `fprintf` require a format string containing ordinary characters and/or conversion specifications. Ordinary characters are printed as is; conversion specifications describe how the remaining arguments are to be converted to character form for display. Section 3.1 described conversion specifications briefly, and we added more details in later chapters. We'll now review what we know about conversion specifications and fill in the remaining gaps.

A `...printf` conversion specification consists of the `%` character, followed by as many as five distinct items:



Here's a detailed description of these items, which must appear in the order shown:

- ***Flags*** (optional; more than one permitted). The - flag causes left justification within a field; the other flags affect the way numbers are displayed. Table 22.4 gives a complete list of flags.

Table 22.4 Flags for ...printf Functions

Flag	*Meaning*
-	Left-justify within field. (The default is right justification.)
+	Numbers produced by signed conversions always begin with + or -. (Normally, only negative numbers are preceded by a sign.)
space	Nonnegative numbers produced by signed conversions are preceded by a space. (The + flag overrides the *space* flag.)
#	Octal numbers begin with `0`, nonzero hexadecimal numbers with `0x` or `0X`. Floating-point numbers always have a decimal point. Trailing zeros aren't removed from numbers printed with the `g` or `G` conversions.
`0` *(zero)*	Numbers are padded with leading zeros up to the field width. The `0` flag is ignored if the conversion is `d`, `i`, `o`, `u`, `x`, or `X` and a precision is specified. (The - flag overrides the `0` flag.)

- ***Minimum field width*** (optional). An item that's too small to occupy this number of characters will be padded. (By default, spaces are added to the left of the item, thus right-justifying it within the field.) An item that's too large for the field width will still be displayed in its entirety. The field width is either an integer or the character *. If * is present, the field width is obtained from the next argument. If this argument is negative, it's treated as a positive number preceded by a - flag.
- ***Precision*** (optional). The meaning of the precision depends on the conversion:

  `d`, `i`, `o`, `u`, `x`, `X`: minimum number of digits (leading zeros are added if the number has fewer digits)
  `a`, `A`, `e`, `E`, `f`, `F`: number of digits after the decimal point
  `g`, `G`: number of significant digits
  `s`: maximum number of bytes

  The precision is a period (.) followed by an integer or the character *. If * is present, the precision is obtained from the next argument. (If this argument is negative, the effect is the same as not specifying a precision.) If only the period is present, the precision is zero.

- ***Length modifier*** (optional). The presence of a length modifier indicates that the item to be displayed has a type that's longer or shorter than is normal for a particular conversion specification. (For example, `%d` normally refers to an `int` value; `%hd` is used to display a `short int` and `%ld` is used to display a `long int`.) Table 22.5 lists each length modifier, the conversion specifiers with which it may be used, and the type indicated by the combination of the two. (Any combination of length modifier and conversion specifier not shown in the table causes undefined behavior.)

Table 22.5 Length Modifiers for ...`printf` Functions

Length Modifier	*Conversion Specifiers*	*Meaning*
`hh`†	`d, i, o, u, x, X`	`signed char, unsigned char`
	`n`	`signed char *`
`h`	`d, i, o, u, x, X`	`short int, unsigned short int`
	`n`	`short int *`
`l` *(ell)*	`d, i, o, u, x, X`	`long int, unsigned long int`
	`n`	`long int *`
	`c`	`wint_t`
	`s`	`wchar_t *`
	`a, A, e, E, f, F, g, G`	no effect
`ll`† *(ell-ell)*	`d, i, o, u, x, X`	`long long int, unsigned long long int`
	`n`	`long long int *`
`j`†	`d, i, o, u, x, X`	`intmax_t, uintmax_t`
	`n`	`intmax_t *`
`z`†	`d, i, o, u, x, X`	`size_t`
	`n`	`size_t *`
`t`†	`d, i, o, u, x, X`	`ptrdiff_t`
	`n`	`ptrdiff_t *`
`L`	`a, A, e, E, f, F, g, G`	`long double`

†C99 only

- ***Conversion specifier***. The conversion specifier must be one of the characters listed in Table 22.6. Notice that `f`, `F`, `e`, `E`, `g`, `G`, `a`, and `A` are all designed to write `double` values. However, they work fine with `float` values as well; thanks to the default argument promotions, `float` arguments are converted automatically to `double` when passed to a function with a variable number of arguments. Similarly, a character passed to ...`printf` is converted automatically to `int`, so the `c` conversion works properly.

default argument promotions ➤9.3



Be careful to follow the rules described here; the effect of using an invalid conversion specification is undefined.

Table 22.6 Conversion Specifiers for ...printf Functions

Conversion Specifier	*Meaning*
d, i	Converts an int value to decimal form.
o, u, x, X	Converts an unsigned int value to base 8 (o), base 10 (u), or base 16 (x, X). x displays the hexadecimal digits a–f in lower case; X displays them in upper case.
f, F†	Converts a double value to decimal form, putting the decimal point in the correct position. If no precision is specified, displays six digits after the decimal point.
e, E	Converts a double value to scientific notation. If no precision is specified, displays six digits after the decimal point. If e is chosen, the exponent is preceded by the letter e; if E is chosen, the exponent is preceded by E.
g, G	g converts a double value to either f form or e form. e form is selected if the number's exponent is less than –4 *or* greater than or equal to the precision. Trailing zeros are not displayed (unless the # flag is used); a decimal point appears only when followed by a digit. G chooses between F and E forms.
a†, A†	Converts a double value to hexadecimal scientific notation using the form [-]0x*h*.*hhhh*p±*d*, where [-] is an optional minus sign, the *h*'s represent hex digits, ± is either a plus or minus sign, and *d* is the exponent. *d* is a decimal number that represents a power of 2. If no precision is specified, enough digits are displayed after the decimal point to represent the exact value of the number (if possible). a displays the hex digits a–f in lower case; A displays them in upper case. The choice of a or A also affects the case of the letters x and p.
c	Displays an int value as an unsigned character.
s	Writes the characters pointed to by the argument. Stops writing when the number of bytes specified by the precision (if present) is reached or a null character is encountered.
p	Converts a void * value to printable form.
n	The corresponding argument must point to an object of type int. Stores in this object the number of characters written so far by this call of ...printf; produces no output.
%	Writes the character %.

†C99 only

C99 C99 Changes to ...printf Conversion Specifications

The conversion specifications for printf and fprintf have undergone a number of changes in C99:

- ***Additional length modifiers.*** C99 adds the hh, ll, j, z, and t length modifiers. hh and ll provide additional length options, j allows greatest-width integers to be written, and z and t make it easier to write values of type size_t and ptrdiff_t, respectively.

greatest-width integers ➤27.1

- ***Additional conversion specifiers.*** C99 adds the `F`, `a`, and `A` conversion specifiers. `F` is the same as `f` except for the way in which infinity and NaN (see below) are written. The `a` and `A` conversion specifications are rarely used. They're related to hexadecimal floating constants, which are discussed in the Q&A section at the end of Chapter 7.

IEEE floating-point standard ➤23.4

- ***Ability to write infinity and NaN.*** The IEEE 754 floating-point standard allows the result of a floating-point operation to be infinity, negative infinity, or NaN ("not a number"). For example, dividing 1.0 by 0.0 yields positive infinity, dividing –1.0 by 0.0 yields negative infinity, and dividing 0.0 by 0.0 yields NaN (because the result is mathematically undefined). In C99, the `a`, `A`, `e`, `E`, `f`, `F`, `g`, and `G` conversion specifiers are capable of converting these special values to a form that can be displayed. `a`, `e`, `f`, and `g` convert positive infinity to `inf` or `infinity` (either one is legal), negative infinity to `-inf` or `-infinity`, and NaN to `nan` or `-nan` (possibly followed by a series of characters enclosed in parentheses). `A`, `E`, `F`, and `G` are equivalent to `a`, `e`, `f`, and `g`, except that upper-case letters are used (`INF`, `INFINITY`, `NAN`).

wide characters ➤25.2

- ***Support for wide characters.*** Another C99 feature is the ability of `fprintf` to write wide characters. The `%lc` conversion specification is used to write a single wide character; `%ls` is used for a string of wide characters.
- ***Previously undefined conversion specifications now allowed.*** In C89, the effect of using `%le`, `%lE`, `%lf`, `%lg`, and `%lG` is undefined. These conversion specifications are legal in C99 (the `l` length modifier is simply ignored).

Examples of ...printf Conversion Specifications

Whew! It's about time for a few examples. We've seen plenty of everyday conversion specifications in previous chapters, so we'll concentrate here on illustrating some of the more advanced ones. As in previous chapters, I'll use • to represent the space character.

Let's start off by examining the effect of flags on the `%d` conversion (they have a similar effect on other conversions). The first line of Table 22.7 shows the effect of `%8d` without any flags. The next four lines show the effect of the `-`, `+`, *space*, and `0` flags (the # flag is never used with `%d`). The remaining lines show the effect of combinations of flags.

Table 22.7 Effect of Flags on the `%d` Conversion

Conversion Specification	*Result of Applying Conversion to 123*	*Result of Applying Conversion to –123*
`%8d`	`•••••123`	`••••-123`
`%-8d`	`123•••••`	`-123••••`
`%+8d`	`••••+123`	`••••-123`
`% 8d`	`•••••123`	`••••-123`
`%08d`	`00000123`	`-0000123`
`%-+8d`	`+123••••`	`-123••••`
`%- 8d`	`•123••••`	`-123••••`
`%+08d`	`+0000123`	`-0000123`
`% 08d`	`•0000123`	`-0000123`

Table 22.8 shows the effect of the # flag on the o, x, X, g, and G conversions.

Table 22.8
Effect of the # Flag

Conversion Specification	*Result of Applying Conversion to 123*	*Result of Applying Conversion to 123.0*
%8o	•••••173	
%#8o	••••0173	
%8x	••••••7b	
%#8x	••••0x7b	
%8X	••••••7B	
%#8X	••••0X7B	
%8g		•••••123
%#8g		•123.000
%8G		•••••123
%#8G		•123.000

In previous chapters, we've used the minimum field width and precision when displaying numbers, so there's no point in more examples here. Instead, Table 22.9 shows the effect of the minimum field width and precision on the %s conversion.

Table 22.9
Effect of Minimum Field Width and Precision on the %s Conversion

Conversion Specification	*Result of Applying Conversion to* "bogus"	*Result of Applying Conversion to* "buzzword"
%6s	•bogus	buzzword
%-6s	bogus•	buzzword
%.4s	bogu	buzz
%6.4s	••bogu	••buzz
%-6.4s	bogu••	buzz••

Table 22.10 illustrates how the %g conversion displays some numbers in %e form and others in %f form. All numbers in the table were written using the %.4g conversion specification. The first two numbers have exponents of at least 4, so they're displayed in %e form. The next eight numbers are displayed in %f form. The last two numbers have exponents less than –4, so they're displayed in %e form.

Table 22.10
Examples of the %g Conversion

Number	*Result of Applying %.4g Conversion to Number*
123456.	1.235e+05
12345.6	1.235e+04
1234.56	1235
123.456	123.5
12.3456	12.35
1.23456	1.235
.123456	0.1235
.0123456	0.01235
.00123456	0.001235
.000123456	0.0001235
.0000123456	1.235e-05
.00000123456	1.235e-06

In the past, we've assumed that the minimum field width and precision were constants embedded in the format string. Putting the * character where either number would normally go allows us to specify it as an argument *after* the format string. For example, the following calls of `printf` all produce the same output:

```
printf("%6.4d", i);
printf("%*.4d", 6, i);
printf("%6.*d", 4, i);
printf("%*.*d", 6, 4, i);
```

Notice that the values to be filled in for the * come just before the value to be displayed. A major advantage of *, by the way, is that it allows us to use a macro to specify the width or precision:

```
printf("%*d", WIDTH, i);
```

We can even compute the width or precision during program execution:

```
printf("%*d", page_width / num_cols, i);
```

The most unusual specifications are `%p` and `%n`. The `%p` conversion allows us to print the value of a pointer:

```
printf("%p", (void *) ptr);   /* displays value of ptr */
```

Although `%p` is occasionally useful during debugging, it's not a feature that most programmers use on a daily basis. The C standard doesn't specify what a pointer looks like when printed using `%p`, but it's likely to be shown as an octal or hexadecimal number.

The `%n` conversion is used to find out how many characters have been printed so far by a call of ...`printf`. For example, after the call

```
printf("%d%n\n", 123, &len);
```

the value of `len` will be 3, since `printf` had written 3 characters (`123`) by the time it reached `%n`. Notice that `&` must precede `len` (because `%n` requires a pointer) and that `len` itself isn't printed.

The ...scanf Functions

```
int fscanf(FILE * restrict stream,
           const char * restrict format, ...);
int scanf(const char * restrict format, ...);
```

fscanf
scanf

`fscanf` and `scanf` read data items from an input stream, using a format string to indicate the layout of the input. After the format string, any number of pointers—each pointing to an object—follow as additional arguments. Input items are converted (according to conversion specifications in the format string) and stored in these objects.

`scanf` always reads from `stdin` (the standard input stream), whereas `fscanf` reads from the stream indicated by its first argument:

```
scanf("%d%d", &i, &j);          /* reads from stdin */
fscanf(fp, "%d%d", &i, &j);     /* reads from fp */
```

A call of `scanf` is equivalent to a call of `fscanf` with `stdin` as the first argument.

The ...`scanf` functions return prematurely if an ***input failure*** occurs (no more input characters could be read) or if a ***matching failure*** occurs (the input characters didn't match the format string). (In C99, an input failure can also occur because of an ***encoding error,*** which means that an attempt was made to read a multibyte character, but the input characters didn't correspond to any valid multibyte character.) Both functions return the number of data items that were read and assigned to objects; they return `EOF` if an input failure occurs before any data items can be read.

C99

multibyte characters ➤25.2

Loops that test `scanf`'s return value are common in C programs. The following loop, for example, reads a series of integers one by one, stopping at the first sign of trouble:

idiom

```
while (scanf("%d", &i) == 1) {
  ...
}
```

...`scanf` Format Strings

Calls of the ...`scanf` functions resemble those of the ...`printf` functions. That similarity can be misleading, however; the ...`scanf` functions work quite differently from the ...`printf` functions. It pays to think of `scanf` and `fscanf` as "pattern-matching" functions. The format string represents a pattern that a ...`scanf` function attempts to match as it reads input. If the input doesn't match the format string, the function returns as soon as it detects the mismatch; the input character that didn't match is "pushed back" to be read in the future.

A ...`scanf` format string may contain three things:

- ***Conversion specifications.*** Conversion specifications in a ...`scanf` format string resemble those in a ...`printf` format string. Most conversion specifications skip white-space characters at the beginning of an input item (the exceptions are `%[`, `%c`, and `%n`). Conversion specifications never skip *trailing* white-space characters, however. If the input contains •`123`¤, the `%d` conversion specification consumes •, `1`, `2`, and `3`, but leaves ¤ unread. (I'm using • to represent the space character and ¤ to represent the new-line character.)

white-space characters ➤3.2

- ***White-space characters.*** One or more consecutive white-space characters in a ...`scanf` format string match zero or more white-space characters in the input stream.
- ***Non-white-space characters.*** A non-white-space character other than `%` matches the same character in the input stream.

For example, the format string `"ISBN %d-%d-%ld-%d"` specifies that the input will consist of:

the letters `ISBN`
possibly some white-space characters
an integer
the - character
an integer (possibly preceded by white-space characters)
the - character
a long integer (possibly preceded by white-space characters)
the - character
an integer (possibly preceded by white-space characters)

...scanf Conversion Specifications

Conversion specifications for ...`scanf` functions are actually a little simpler than those for ...`printf` functions. A ...`scanf` conversion specification consists of the character `%` followed by the items listed below (in the order shown).

- `*` (optional). The presence of `*` signifies ***assignment suppression:*** an input item is read but not assigned to an object. Items matched using `*` aren't included in the count that ...`scanf` returns.
- ***Maximum field width*** (optional). The maximum field width limits the number of characters in an input item; conversion of the item ends if this number is reached. White-space characters skipped at the beginning of a conversion don't count.
- ***Length modifier*** (optional). The presence of a length modifier indicates that the object in which the input item will be stored has a type that's longer or shorter than is normal for a particular conversion specification. Table 22.11 lists each length modifier, the conversion specifiers with which it may be used, and the type indicated by the combination of the two. (Any combination of length modifier and conversion specifier not shown in the table causes undefined behavior.)

Table 22.11 Length Modifiers for ...`scanf` Functions

Length Modifier	*Conversion Specifiers*	*Meaning*
`hh`†	`d, i, o, u, x, X, n`	`signed char *, unsigned char *`
`h`	`d, i, o, u, x, X, n`	`short int *, unsigned short int *`
`l` *(ell)*	`d, i, o, u, x, X, n`	`long int *, unsigned long int *`
	`a, A, e, E, f, F, g, G`	`double *`
	`c, s, or [`	`wchar_t *`
`ll`† *(ell-ell)*	`d, i, o, u, x, X, n`	`long long int *, unsigned long long int *`
`j`†	`d, i, o, u, x, X, n`	`intmax_t *, uintmax_t *`
`z`†	`d, i, o, u, x, X, n`	`size_t *`
`t`†	`d, i, o, u, x, X, n`	`ptrdiff_t *`
`L`	`a, A, e, E, f, F, g, G`	`long double *`

†C99 only

- ***Conversion specifier.*** The conversion specifier must be one of the characters listed in Table 22.12.

Table 22.12 Conversion Specifiers for ...scanf Functions

Conversion Specifier	*Meaning*
d	Matches a decimal integer; the corresponding argument is assumed to have type int *.
i	Matches an integer; the corresponding argument is assumed to have type int *. The integer is assumed to be in base 10 unless it begins with 0 (indicating octal) or with 0x or 0X (hexadecimal).
o	Matches an octal integer; the corresponding argument is assumed to have type unsigned int *.
u	Matches a decimal integer; the corresponding argument is assumed to have type unsigned int *.
x, X	Matches a hexadecimal integer; the corresponding argument is assumed to have type unsigned int *.
a†, A†, e, E, f, F†, g, G	Matches a floating-point number; the corresponding argument is assumed to have type float *. In C99, the number can be infinity or NaN.
c	Matches *n* characters, where *n* is the maximum field width, or one character if no field width is specified. The corresponding argument is assumed to be a pointer to a character array (or a character object, if no field width is specified). Doesn't add a null character at the end.
s	Matches a sequence of non-white-space characters, then adds a null character at the end. The corresponding argument is assumed to be a pointer to a character array.
[	Matches a nonempty sequence of characters from a scanset, then adds a null character at the end. The corresponding argument is assumed to be a pointer to a character array.
p	Matches a pointer value in the form that ...printf would have written it. The corresponding argument is assumed to be a pointer to a void * object.
n	The corresponding argument must point to an object of type int. Stores in this object the number of characters read so far by this call of ...scanf. No input is consumed and the return value of ...scanf isn't affected.
%	Matches the character %.

†C99 only

Numeric data items can always begin with a sign (+ or -). The o, u, x, and X specifiers convert the item to unsigned form, however, so they're not normally used to read negative numbers.

The [specifier is a more complicated (and more flexible) version of the s specifier. A complete conversion specification using [has the form %[*set*] or %[^*set*], where *set* can be any set of characters. (If] is one of the characters in *set*, however, it must come first.) %[*set*] matches any sequence of characters in *set* (the ***scanset***). %[^*set*] matches any sequence of characters *not* in *set* (in other words, the scanset consists of all characters not in *set*). For example, %[abc]

matches any string containing only the letters `a`, `b`, and `c`, while `%[^abc]` matches any string that doesn't contain `a`, `b`, or `c`.

numeric conversion functions ➤ *26.2*

Many of the ...`scanf` conversion specifiers are closely related to the numeric conversion functions in `<stdlib.h>`. These functions convert strings (like `"-297"`) to their equivalent numeric values (–297). The `d` specifier, for example, looks for an optional `+` or `-` sign, followed by a series of decimal digits; this is exactly the same form that the `strtol` function requires when asked to convert a string to a decimal number. Table 22.13 shows the correspondence between conversion specifiers and numeric conversion functions.

Table 22.13 Correspondence between ...`scanf` Conversion Specifiers and Numeric Conversion Functions

Conversion Specifier	*Numeric Conversion Function*
`d`	`strtol` with 10 as the base
`i`	`strtol` with 0 as the base
`o`	`strtoul` with 8 as the base
`u`	`strtoul` with 10 as the base
`x`, `X`	`strtoul` with 16 as the base
`a`, `A`, `e`, `E`, `f`, `F`, `g`, `G`	`strtod`



It pays to be careful when writing calls of `scanf`. An invalid conversion specification in a `scanf` format string is just as bad as one in a `printf` format string; either one causes undefined behavior.

C99 C99 Changes to ...`scanf` Conversion Specifications

The conversion specifications for `scanf` and `fscanf` have undergone some changes in C99, but the list isn't as extensive as it was for the ...`printf` functions:

- ***Additional length modifiers.*** C99 adds the `hh`, `ll`, `j`, `z`, and `t` length modifiers. These correspond to the length modifiers in ...`printf` conversion specifications.
- ***Additional conversion specifiers.*** C99 adds the `F`, `a`, and `A` conversion specifiers. They're provided for symmetry with ...`printf`; the ...`scanf` functions treat them the same as `e`, `E`, `f`, `g`, and `G`.
- ***Ability to read infinity and NaN.*** Just as the ...`printf` functions can write infinity and NaN, the ...`scanf` functions can read these values. To be read properly, they should have the same appearance as values written by the ...`printf` functions, with case being ignored. (For example, either `INF` or `inf` will be read as infinity.)
- ***Support for wide characters.*** The ...`scanf` functions are able to read multibyte characters, which are then converted to wide characters for storage. The `%lc` conversion specification is used to read a single multibyte character or a

sequence of multibyte characters; `%ls` is used to read a string of multibyte characters (a null character is added at the end). The `%l` [*set*] and `%l` [^*set*] conversion specifications can also read a string of multibyte characters.

`scanf` Examples

The next three tables contain sample calls of `scanf`. Each call is applied to the input characters shown to its right. Characters printed in ~~strikeout~~ are consumed by the call. The values of the variables after the call appear to the right of the input.

The examples in Table 22.14 show the effect of combining conversion specifications, white-space characters, and non-white-space characters. In three cases no value is assigned to `j`, so it retains its value from before the call of `scanf`. The examples in Table 22.15 show the effect of assignment suppression and specifying a field width. The examples in Table 22.16 illustrate the more esoteric conversion specifiers (`i`, `[`, and `n`).

Table 22.14 `scanf` Examples (Group 1)

`scanf` *Call*	*Input*	*Variables*
`n = scanf("%d%d", &i, &j);`	~~12~~•,•34¤	n: 1 i: 12 j: unchanged
`n = scanf("%d,%d", &i, &j);`	~~12~~•,•34¤	n: 1 i: 12 j: unchanged
`n = scanf("%d ,%d", &i, &j);`	~~12•,•34¤~~	n: 2 i: 12 j: 34
`n = scanf("%d, %d", &i, &j);`	~~12~~•,•34¤	n: 1 i: 12 j: unchanged

Table 22.15 `scanf` Examples (Group 2)

`scanf` *Call*	*Input*	*Variables*
`n = scanf("%*d%d", &i);`	~~12•34¤~~	n: 1 i: 34
`n = scanf("%*s%s", str);`	~~My•Fair~~•Lady¤	n: 1 str: "Fair"
`n = scanf("%1d%2d%3d", &i, &j, &k);`	~~12345¤~~	n: 3 i: 1 j: 23 k: 45
`n = scanf("%2d%2s%2d", &i, str, &j);`	~~123456¤~~	n: 3 i: 12 str: "34" j: 56

Table 22.16 scanf Examples (Group 3)

scanf Call	*Input*	*Variables*
`n = scanf("%i%i%i", &i, &j, &k);`	~~12•012•0x12~~␤	n: 3 i: 12 j: 10 k: 18
`n = scanf("%[0123456789]", str);`	~~123~~abc␤	n: 1 str: "123"
`n = scanf("%[0123456789]", str);`	abc123␤	n: 0 str: unchanged
`n = scanf("%[^0123456789]", str);`	~~abc~~123␤	n: 1 str: "abc"
`n = scanf("%*d%d%n", &i, &j);`	~~10•20•~~30␤	n: 1 i: 20 j: 5

Detecting End-of-File and Error Conditions

```
void clearerr(FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
```

If we ask a ...`scanf` function to read and store n data items, we expect its return value to be n. If the return value is less than n, something went wrong. There are three possibilities:

- ***End-of-file.*** The function encountered end-of-file before matching the format string completely.
- ***Read error.*** The function was unable to read characters from the stream.
- ***Matching failure.*** A data item was in the wrong format. For example, the function might have encountered a letter while searching for the first digit of an integer.

But how can we tell which kind of failure occurred? In many cases, it doesn't matter; something went wrong, and we've got to abandon the program. There may be times, however, when we'll need to pinpoint the reason for the failure.

Every stream has two indicators associated with it: an ***error indicator*** and an ***end-of-file indicator.*** These indicators are cleared when the stream is opened. Not surprisingly, encountering end-of-file sets the end-of-file indicator, and a read error sets the error indicator. (The error indicator is also set when a write error occurs on an output stream.) A matching failure doesn't change either indicator.

clearerr

Once the error or end-of-file indicator is set, it remains in that state until it's explicitly cleared, perhaps by a call of the `clearerr` function. `clearerr` clears both the end-of-file and error indicators:

```
clearerr(fp);    /* clears eof and error indicators for fp */
```

`clearerr` isn't needed often, since some of the other library functions clear one or both indicators as a side effect.

Q&A

feof
ferror

We can call the `feof` and `ferror` functions to test a stream's indicators to determine why a prior operation on the stream failed. The call `feof(fp)` returns a nonzero value if the end-of-file indicator is set for the stream associated with `fp`. The call `ferror(fp)` returns a nonzero value if the error indicator is set. Both functions return zero otherwise.

Q&A

When `scanf` returns a smaller-than-expected value, we can use `feof` and `ferror` to determine the reason. If `feof` returns a nonzero value, we've reached the end of the input file. If `ferror` returns a nonzero value, a read error occurred during input. If neither returns a nonzero value, a matching failure must have occurred. Regardless of what the problem was, the return value of `scanf` tells us how many data items were read before the problem occurred.

To see how `feof` and `ferror` might be used, let's write a function that searches a file for a line that begins with an integer. Here's how we intend to call the function:

```
n = find_int("foo");
```

`"foo"` is the name of the file to be searched. The function returns the value of the integer that it finds, which is then assigned to `n`. If a problem arises—the file can't be opened, a read error occurs, or no line begins with an integer—`find_int` will return an error code (–1, –2, or –3, respectively). I'll assume that no line in the file begins with a negative integer.

```
int find_int(const char *filename)
{
  FILE *fp = fopen(filename, "r");
  int n;

  if (fp == NULL)
    return -1;                  /* can't open file */

  while (fscanf(fp, "%d", &n) != 1) {
    if (ferror(fp)) {
      fclose(fp);
      return -2;                /* read error */
    }
    if (feof(fp)) {
      fclose(fp);
      return -3;                /* integer not found */
    }
    fscanf(fp, "%*[^\n]");      /* skips rest of line */
  }

  fclose(fp);
  return n;
}
```

The `while` loop's controlling expression calls `fscanf` in an attempt to read an integer from the file. If the attempt fails (`fscanf` returns a value other than 1),

`find_int` calls `ferror` and `feof` to see if the problem was a read error or end-of-file. If not, `fscanf` must have failed because of a matching error, so `find_int` skips the rest of the characters on the current line and tries again. Note the use of the conversion `%*[^\n]` to skip all characters up to the next new-line. (Now that we know about scansets, it's time to show off!)

22.4 Character I/O

In this section, we'll examine library functions that read and write single characters. These functions work equally well with text streams and binary streams.

You'll notice that the functions in this section treat characters as values of type `int`, not `char`. One reason is that the input functions indicate an end-of-file (or error) condition by returning `EOF`, which is a negative integer constant.

Output Functions

```
int fputc(int c, FILE *stream);
int putc(int c, FILE *stream);
int putchar(int c);
```

putchar

`putchar` writes one character to the `stdout` stream:

```
putchar(ch);      /* writes ch to stdout */
```

fputc
putc

`fputc` and `putc` are more general versions of `putchar` that write a character to an arbitrary stream:

```
fputc(ch, fp);    /* writes ch to fp */
putc(ch, fp);     /* writes ch to fp */
```

Although `putc` and `fputc` do the same thing, `putc` is usually implemented as a macro (as well as a function), while `fputc` is implemented only as a function. `putchar` itself is usually a macro defined in the following way:

```
#define putchar(c) putc((c), stdout)
```

It may seem odd that the library provides both `putc` and `fputc`. But, as we saw in Section 14.3, macros have several potential problems. The C standard allows the `putc` macro to evaluate the `stream` argument more than once, which `fputc` isn't permitted to do. Although programmers usually prefer `putc`, which gives a faster program, `fputc` is available as an alternative.

Q&A

If a write error occurs, all three functions set the error indicator for the stream and return `EOF`; otherwise, they return the character that was written.

Input Functions

```
int fgetc(FILE *stream);
int getc(FILE *stream);
int getchar(void);
int ungetc(int c, FILE *stream);
```

getchar `getchar` reads a character from the `stdin` stream:

```
ch = getchar();    /* reads a character from stdin */
```

fgetc
getc `fgetc` and `getc` read a character from an arbitrary stream:

```
ch = fgetc(fp);    /* reads a character from fp */
ch = getc(fp);     /* reads a character from fp */
```

All three functions treat the character as an `unsigned char` value (which is then converted to `int` type before it's returned). As a result, they never return a negative value other than `EOF`.

The relationship between `getc` and `fgetc` is similar to that between `putc` and `fputc`. `getc` is usually implemented as a macro (as well as a function), while `fgetc` is implemented only as a function. `getchar` is normally a macro as well:

```
#define getchar() getc(stdin)
```

For reading characters from a file, programmers usually prefer `getc` over `fgetc`. Since `getc` is normally available in macro form, it tends to be faster. `fgetc` can be used as a backup if `getc` isn't appropriate. (The standard allows the `getc` macro to evaluate its argument more than once, which may be a problem.)

The `fgetc`, `getc`, and `getchar` functions behave the same if a problem occurs. At end-of-file, they set the stream's end-of-file indicator and return `EOF`. If a read error occurs, they set the stream's error indicator and return `EOF`. To differentiate between the two situations, we can call either `feof` or `ferror`.

One of the most common uses of `fgetc`, `getc`, and `getchar` is to read characters from a file, one by one, until end-of-file occurs. It's customary to use the following `while` loop for that purpose:

idiom
```
while ((ch = getc(fp)) != EOF) {
  ...
}
```

After reading a character from the file associated with `fp` and storing it in the variable `ch` (which must be of type `int`), the `while` test compares `ch` with `EOF`. If `ch` isn't equal to `EOF`, we're not at the end of the file yet, so the body of the loop is executed. If `ch` is equal to `EOF`, the loop terminates.



Always store the return value of `fgetc`, `getc`, or `getchar` in an `int` variable, not a `char` variable. Testing a `char` variable against `EOF` may give the wrong result.

ungetc

There's one other character input function, `ungetc`, which "pushes back" a character read from a stream and clears the stream's end-of-file indicator. This capability can be handy if we need a "lookahead" character during input. For instance, to read a series of digits, stopping at the first nondigit, we could write

isdigit function ➤23.5

```
while (isdigit(ch = getc(fp))) {
  ...
}
ungetc(ch, fp);   /* pushes back last character read */
```

The number of characters that can be pushed back by consecutive calls of `ungetc`—with no intervening read operations—depends on the implementation and the type of stream involved; only the first call is guaranteed to succeed. Calling a file-positioning function (`fseek`, `fsetpos`, or `rewind`) causes the pushed-back characters to be lost.

file-positioning functions ➤22.7

`ungetc` returns the character it was asked to push back. However, it returns `EOF` if an attempt is made to push back `EOF` or to push back more characters than the implementation allows.

PROGRAM

Copying a File

The following program makes a copy of a file. The names of the original file and the new file will be specified on the command line when the program is executed. For example, to copy the file `f1.c` to `f2.c`, we'd use the command

```
fcopy f1.c f2.c
```

`fcopy` will issue an error message if there aren't exactly two file names on the command line or if either file can't be opened.

fcopy.c

```
/* Copies a file */

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
  FILE *source_fp, *dest_fp;
  int ch;
```

```
  if (argc != 3) {
    fprintf(stderr, "usage: fcopy source dest\n");
    exit(EXIT_FAILURE);
  }

  if ((source_fp = fopen(argv[1], "rb")) == NULL) {
    fprintf(stderr, "Can't open %s\n", argv[1]);
    exit(EXIT_FAILURE);
  }

  if ((dest_fp = fopen(argv[2], "wb")) == NULL) {
    fprintf(stderr, "Can't open %s\n", argv[2]);
    fclose(source_fp);
    exit(EXIT_FAILURE);
  }

  while ((ch = getc(source_fp)) != EOF)
    putc(ch, dest_fp);

  fclose(source_fp);
  fclose(dest_fp);
  return 0;
}
```

Using `"rb"` and `"wb"` as the file modes enables `fcopy` to copy both text and binary files. If we used `"r"` and `"w"` instead, the program wouldn't necessarily be able to copy binary files.

22.5 Line I/O

We'll now turn to library functions that read and write lines. These functions are used mostly with text streams, although it's legal to use them with binary streams as well.

Output Functions

```
int fputs(const char * restrict s,
          FILE * restrict stream);
int puts(const char *s);
```

puts

We encountered the `puts` function in Section 13.3; it writes a string of characters to `stdout`:

```
puts("Hi, there!");   /* writes to stdout */
```

After it writes the characters in the string, `puts` always adds a new-line character.

fputs

`fputs` is a more general version of `puts`. Its second argument indicates the stream to which the output should be written:

```
fputs("Hi, there!", fp);   /* writes to fp */
```

Unlike `puts`, the `fputs` function doesn't write a new-line character unless one is present in the string.

Both functions return `EOF` if a write error occurs; otherwise, they return a nonnegative number.

Input Functions

```
char *fgets(char * restrict s, int n,
            FILE * restrict stream);
char *gets(char *s);
```

gets

The `gets` function, which we first encountered in Section 13.3, reads a line of input from `stdin`:

```
gets(str);   /* reads a line from stdin */
```

`gets` reads characters one by one, storing them in the array pointed to by `str`, until it reads a new-line character (which it discards).

fgets

`fgets` is a more general version of `gets` that can read from any stream. `fgets` is also safer than `gets`, since it limits the number of characters that it will store. Here's how we might use `fgets`, assuming that `str` is the name of a character array:

```
fgets(str, sizeof(str), fp);   /* reads a line from fp */
```

This call will cause `fgets` to read characters until it reaches the first new-line character or `sizeof(str) - 1` characters have been read, whichever happens first. If it reads the new-line character, `fgets` stores it along with the other characters. (Thus, `gets` *never* stores the new-line character, but `fgets` *sometimes* does.)

Both `gets` and `fgets` return a null pointer if a read error occurs or they reach the end of the input stream before storing any characters. (As usual, we can call `feof` or `ferror` to determine which situation occurred.) Otherwise, both return their first argument, which points to the array in which the input was stored. As you'd expect, both functions store a null character at the end of the string.

Now that you know about `fgets`, I'd suggest using it instead of `gets` in most situations. With `gets`, there's always the possibility of stepping outside the bounds of the receiving array, so it's safe to use only when the string being read is *guaranteed* to fit into the array. When there's no guarantee (and there usually isn't), it's much safer to use `fgets`. Note that `fgets` will read from the standard input stream if passed `stdin` as its third argument:

```
fgets(str, sizeof(str), stdin);
```

22.6 Block I/O

```
size_t fread(void * restrict ptr,
             size_t size, size_t nmemb,
             FILE * restrict stream);
size_t fwrite(const void * restrict ptr,
              size_t size, size_t nmemb,
              FILE * restrict stream);
```

The `fread` and `fwrite` functions allow a program to read and write large blocks of data in a single step. `fread` and `fwrite` are used primarily with binary streams, although—with care—it's possible to use them with text streams as well.

Q&A

fwrite

`fwrite` is designed to copy an array from memory to a stream. The first argument in a call of `fwrite` is the array's address, the second argument is the size of each array element (in bytes), and the third argument is the number of elements to write. The fourth argument is a file pointer, indicating where the data should be written. To write the entire contents of the array `a`, for instance, we could use the following call of `fwrite`:

```
fwrite(a, sizeof(a[0]), sizeof(a) / sizeof(a[0]), fp);
```

There's no rule that we have to write the entire array; we could just as easily write any portion of it. `fwrite` returns the number of elements (*not* bytes) actually written. This number will be less than the third argument if a write error occurs.

fread

`fread` will read the elements of an array from a stream. `fread`'s arguments are similar to `fwrite`'s: the array's address, the size of each element (in bytes), the number of elements to read, and a file pointer. To read the contents of a file into the array `a`, we might use the following call of `fread`:

```
n = fread(a, sizeof(a[0]), sizeof(a) / sizeof(a[0]), fp);
```

It's important to check `fread`'s return value, which indicates the actual number of elements (*not* bytes) read. This number should equal the third argument unless the end of the input file was reached or a read error occurred. The `feof` and `ferror` functions can be used to determine the reason for any shortage.



Be careful not to confuse `fread`'s second and third arguments. Consider the following call of `fread`:

```
fread(a, 1, 100, fp)
```

We're asking `fread` to read 100 one-byte elements, so it will return a value

between 0 and 100. The following call asks `fread` to read one block of 100 bytes:

```
fread(a, 100, 1, fp)
```

`fread`'s return value in this case will be either 0 or 1.

`fwrite` is convenient for a program that needs to store data in a file before terminating. Later, the program (or another program, for that matter) can use `fread` to read the data back into memory. Despite appearances, the data doesn't need to be in array form; `fread` and `fwrite` work just as well with variables of all kinds. Structures, in particular, can be read by `fread` or written by `fwrite`. To write a structure variable `s` to a file, for instance, we could use the following call of `fwrite`:

```
fwrite(&s, sizeof(s), 1, fp);
```



Be careful when using `fwrite` to write out structures that contain pointer values; these values aren't guaranteed to be valid when read back in.

22.7 File Positioning

```
int fgetpos(FILE * restrict stream,
            fpos_t * restrict pos);
int fseek(FILE *stream, long int offset, int whence);
int fsetpos(FILE *stream, const fpos_t *pos);
long int ftell(FILE *stream);
void rewind(FILE *stream);
```

Every stream has an associated ***file position.*** When a file is opened, the file position is set at the beginning of the file. (If the file is opened in "append" mode, however, the initial file position may be at the beginning or end of the file, depending on the implementation.) Then, when a read or write operation is performed, the file position advances automatically, allowing us to move through the file in a sequential manner.

Although sequential access is fine for many applications, some programs need the ability to jump around within a file, accessing some data here and other data there. If a file contains a series of records, for example, we might want to jump directly to a particular record and read it or update it. `<stdio.h>` supports this form of access by providing five functions that allow a program to determine the current file position or to change it.

fseek

The `fseek` function changes the file position associated with the first argument (a file pointer). The third argument specifies whether the new position is to

be calculated with respect to the beginning of the file, the current position, or the end of the file. `<stdio.h>` defines three macros for this purpose:

`SEEK_SET`	Beginning of file
`SEEK_CUR`	Current file position
`SEEK_END`	End of file

The second argument is a (possibly negative) byte count. To move to the beginning of a file, for example, the seek direction would be `SEEK_SET` and the byte count would be zero:

```
fseek(fp, 0L, SEEK_SET);      /* moves to beginning of file */
```

To move to the end of a file, the seek direction would be `SEEK_END`:

```
fseek(fp, 0L, SEEK_END);      /* moves to end of file */
```

To move back 10 bytes, the seek direction would be `SEEK_CUR` and the byte count would be –10:

```
fseek(fp, -10L, SEEK_CUR);    /* moves back 10 bytes */
```

Note that the byte count has type `long int`, so I've used `0L` and `-10L` as arguments. (`0` and `-10` would also work, of course, since arguments are converted to the proper type automatically.)

Normally, `fseek` returns zero. If an error occurs (the requested position doesn't exist, for example), `fseek` returns a nonzero value.

The file-positioning functions are best used with binary streams, by the way. C doesn't prohibit programs from using them with text streams, but care is required because of operating system differences. `fseek` in particular is sensitive to whether a stream is text or binary. For text streams, either (1) `offset` (`fseek`'s second argument) must be zero or (2) `whence` (its third argument) must be `SEEK_SET` and `offset` a value obtained by a previous call of `ftell`. (In other words, we can only use `fseek` to move to the beginning or end of a text stream or to return to a place that was visited previously.) For binary streams, `fseek` isn't required to support calls in which `whence` is `SEEK_END`.

ftell

errno variable ➤*24.2*

The `ftell` function returns the current file position as a long integer. (If an error occurs, `ftell` returns `-1L` and stores an error code in `errno`.) The value returned by `ftell` may be saved and later supplied to a call of `fseek`, making it possible to return to a previous file position:

```
long file_pos;
...
file_pos = ftell(fp);              /* saves current position */
...
fseek(fp, file_pos, SEEK_SET);     /* returns to old position */
```

If `fp` is a binary stream, the call `ftell(fp)` returns the current file position as a byte count, where zero represents the beginning of the file. If `fp` is a text stream, however, `ftell(fp)` isn't necessarily a byte count. As a result, it's best not to perform arithmetic on values returned by `ftell`. For example, it's not a good

idea to subtract values returned by `ftell` to see how far apart two file positions are.

rewind

The `rewind` function sets the file position at the beginning. The call `rewind(fp)` is nearly equivalent to `fseek(fp, 0L, SEEK_SET)`. The difference? `rewind` doesn't return a value but does clear the error indicator for `fp`.

fgetpos
fsetpos

Q&A

`fseek` and `ftell` have one problem: they're limited to files whose positions can be stored in a long integer. For working with very large files, C provides two additional functions: `fgetpos` and `fsetpos`. These functions can handle large files because they use values of type `fpos_t` to represent file positions. An `fpos_t` value isn't necessarily an integer; it could be a structure, for instance.

The call `fgetpos(fp, &file_pos)` stores the file position associated with `fp` in the `file_pos` variable. The call `fsetpos(fp, &file_pos)` sets the file position for `fp` to be the value stored in `file_pos`. (This value must have been obtained by a previous call of `fgetpos`.) If a call of `fgetpos` or `fsetpos` fails, it stores an error code in `errno`. Both functions return zero when they succeed and a nonzero value when they fail.

Here's how we might use `fgetpos` and `fsetpos` to save a file position and return to it later:

```
fpos_t file_pos;
...
fgetpos(fp, &file_pos);    /* saves current position */
...
fsetpos(fp, &file_pos);    /* returns to old position */
```

PROGRAM

Modifying a File of Part Records

The following program opens a binary file containing `part` structures, reads the structures into an array, sets the `on_hand` member of each structure to 0, and then writes the structures back to the file. Note that the program opens the file in `"rb+"` mode, allowing both reading and writing.

invclear.c

```
/* Modifies a file of part records by setting the quantity
   on hand to zero for all records */

#include <stdio.h>
#include <stdlib.h>

#define NAME_LEN 25
#define MAX_PARTS 100

struct part {
  int number;
  char name[NAME_LEN+1];
  int on_hand;
} inventory[MAX_PARTS];
```

```
int num_parts;

int main(void)
{
  FILE *fp;
  int i;

  if ((fp = fopen("inventory.dat", "rb+")) == NULL) {
    fprintf(stderr, "Can't open inventory file\n");
    exit(EXIT_FAILURE);
  }

  num_parts = fread(inventory, sizeof(struct part),
                    MAX_PARTS, fp);

  for (i = 0; i < num_parts; i++)
    inventory[i].on_hand = 0;

  rewind(fp);
  fwrite(inventory, sizeof(struct part), num_parts, fp);
  fclose(fp);

  return 0;
}
```

Calling `rewind` is critical, by the way. After the `fread` call, the file position is at the end of the file. If we were to call `fwrite` without calling `rewind` first, `fwrite` would add new data to the end of the file instead of overwriting the old data.

22.8 String I/O

The functions described in this section are a bit unusual, since they have nothing to do with streams or files. Instead, they allow us to read and write data using a string as though it were a stream. The `sprintf` and `snprintf` functions write characters into a string in the same way they would be written to a stream; the `sscanf` function reads characters from a string as though it were reading from a stream. These functions, which closely resemble `printf` and `scanf`, are quite useful. `sprintf` and `snprintf` give us access to `printf`'s formatting capabilities without actually having to write data to a stream. Similarly, `sscanf` gives us access to `scanf`'s powerful pattern-matching capabilities. The remainder of this section covers `sprintf`, `snprintf`, and `sscanf` in detail.

Three similar functions (`vsprintf`, `vsnprintf`, and `vsscanf`) also belong to `<stdio.h>`. However, these functions rely on the `va_list` type, which is declared in `<stdarg.h>`. I'll postpone discussing them until Section 26.1, which covers that header.

Output Functions

```
int sprintf(char * restrict s,
            const char * restrict format, ...);
int snprintf(char * restrict s, size_t n,
             const char * restrict format, ...);
```

Note: In this and subsequent chapters, the prototype for a function that is new in C99 will be in italics. Also, the name of the function will be italicized when it appears in the left margin.

sprintf

The `sprintf` function is similar to `printf` and `fprintf`, except that it writes output into a character array (pointed to by its first argument) instead of a stream. `sprintf`'s second argument is a format string identical to that used by `printf` and `fprintf`. For example, the call

```
sprintf(date, "%d/%d/%d", 9, 20, 2010);
```

will write `"9/20/2010"` into `date`. When it's finished writing into a string, `sprintf` adds a null character and returns the number of characters stored (not counting the null character). If an encoding error occurs (a wide character could not be translated into a valid multibyte character), `sprintf` returns a negative value.

`sprintf` has a variety of uses. For example, we might occasionally want to format data for output without actually writing it. We can use `sprintf` to do the formatting, then save the result in a string until it's time to produce output. `sprintf` is also convenient for converting numbers to character form.

snprintf

The `snprintf` function is the same as `sprintf`, except for the additional parameter `n`. No more than `n` – 1 characters will be written to the string, not counting the terminating null character, which is always written unless `n` is zero. (Equivalently, we could say that `snprintf` writes at most `n` characters to the string, the last of which is a null character.) For example, the call

```
snprintf(name, 13, "%s, %s", "Einstein", "Albert");
```

will write `"Einstein, Al"` into `name`.

`snprintf` returns the number of characters that would have been written (not including the null character) had there been no length restriction. If an encoding error occurs, `snprintf` returns a negative number. To see if `snprintf` had room to write all the requested characters, we can test whether its return value was nonnegative and less than `n`.

Input Functions

```
int sscanf(const char * restrict s,
           const char * restrict format, ...);
```

sscanf

The `sscanf` function is similar to `scanf` and `fscanf`, except that it reads from a string (pointed to by its first argument) instead of reading from a stream. `sscanf`'s second argument is a format string identical to that used by `scanf` and `fscanf`.

`sscanf` is handy for extracting data from a string that was read by another input function. For example, we might use `fgets` to obtain a line of input, then pass the line to `sscanf` for further processing:

```
fgets(str, sizeof(str), stdin);  /* reads a line of input */
sscanf(str, "%d%d", &i, &j);     /* extracts two integers */
```

One advantage of using `sscanf` instead of `scanf` or `fscanf` is that we can examine an input line as many times as needed, not just once, making it easier to recognize alternate input forms and to recover from errors. Consider the problem of reading a date that's written either in the form *month/day/year* or *month-day-year*. Assuming that `str` contains a line of input, we can extract the month, day, and year as follows:

```
if (sscanf(str, "%d /%d /%d", &month, &day, &year) == 3)
  printf("Month: %d, day: %d, year: %d\n", month, day, year);
else if (sscanf(str, "%d -%d -%d", &month, &day, &year) == 3)
  printf("Month: %d, day: %d, year: %d\n", month, day, year);
else
  printf("Date not in the proper form\n");
```

Like the `scanf` and `fscanf` functions, `sscanf` returns the number of data items successfully read and stored. `sscanf` returns `EOF` if it reaches the end of the string (marked by a null character) before finding the first item.

Q & A

Q: If I use input or output redirection, will the redirected file names show up as command-line arguments? [p. 541]

A: No; the operating system removes them from the command line. Let's say that we run a program by entering

```
demo foo <in file bar >out file baz
```

The value of `argc` will be 4, `argv[0]` will point to the program name, `argv[1]` will point to `"foo"`, `argv[2]` will point to `"bar"`, and `argv[3]` will point to `"baz"`.

Q: I thought that the end of a line was always marked by a new-line character. Now you're saying that the end-of-line marker varies, depending on the operating system. How you explain this discrepancy? [p. 542]

A: C library functions make it *appear* as though each line ends with a single new-line

character. Regardless of whether an input file contains a carriage-return character, a line-feed character, or both, a library function such as `getc` will return a single new-line character. The output functions perform the reverse translation. If a program calls a library function to write a new-line character to a file, the function will translate the character into the appropriate end-of-line marker. C's approach makes programs more portable and easier to write; we can work with text files without having to worry about how end-of-line is actually represented. Note that input/output performed on a file opened in binary mode isn't subject to any character translation—carriage return and line feed are treated the same as the other characters.

Q: I'm writing a program that needs to save data in a file, to be read later by another program. Is it better to store the data in text form or binary form? [p. 542]

A: That depends. If the data is all text to start with, there's not much difference. If the data contains numbers, however, the decision is tougher.

Binary form is usually preferable, since it can be read and written quickly. Numbers are already in binary form when stored in memory, so copying them to a file is easy. Writing numbers in text form is much slower, since each number must be converted (usually by `fprintf`) to character form. Reading the file later will also take more time, since numbers will have to be converted from text form back to binary. Moreover, storing data in binary form often saves space, as we saw in Section 22.1.

Binary files have two disadvantages, however. They're hard for humans to read, which can hamper debugging. Also, binary files generally aren't portable from one system to another, since different kinds of computers store data in different ways. For instance, some machines store `int` values using two bytes but others use four bytes. There's also the issue of byte order (big-endian versus little-endian).

Q: C programs for UNIX never seem to use the letter `b` in the mode string, even when the file being opened is binary. What gives? [p. 544]

A: In UNIX, text files and binary files have exactly the same format, so there's never any need to use `b`. UNIX programmers should still include the `b`, however, so that their programs will be more portable to other operating systems.

Q: I've seen programs that call `fopen` and put the letter `t` in the mode string. What does `t` mean?

A: The C standard allows additional characters to appear in the mode string, provided that they follow `r`, `w`, `a`, `b`, or `+`. Some compilers allow the use of `t` to indicate that a file is to be opened in text mode instead of binary mode. Of course, text mode is the default anyway, so `t` adds nothing. Whenever possible, it's best to avoid using `t` and other nonportable features.

Q: Why bother to call `fclose` to close a file? Isn't it true that all open files are closed automatically when a program terminates? [p. 545]

A: That's usually true, but not if the program calls `abort` to terminate. Even when `abort` isn't used, though, there are still good reasons to call `fclose`. First, it reduces the number of open files. Operating systems limit the number of files that a program may have open at the same time; large programs may bump into this limit. (The macro `FOPEN_MAX`, defined in `<stdio.h>`, specifies the minimum number of files that the implementation guarantees can be open simultaneously.) Second, the program becomes easier to understand and modify; by looking for the call of `fclose`, it's easier for the reader to determine the point at which a file is no longer in use. Third, there's the issue of safety. Closing a file ensures that its contents and directory entry are updated properly; if the program should crash later, at least the file will be intact.

`abort` function ➤26.2

Q: I'm writing a program that will prompt the user to enter a file name. How long should I make the character array that will store the file name? [p. 546]

A: That depends on your operating system. Fortunately, you can use the macro `FILENAME_MAX` (defined in `<stdio.h>`) to specify the size of the array. `FILENAME_MAX` is the length of a string that will hold the longest file name that the implementation guarantees can be opened.

Q: Can `fflush` flush a stream that was opened for both reading and writing? [p. 549]

A: According to the C standard, the effect of calling `fflush` is defined for a stream that (a) was opened for output, or (b) was opened for updating and whose last operation was not a read. In all other cases, the effect of calling `fflush` is undefined. When `fflush` is passed a null pointer, it flushes all streams that satisfy either (a) or (b).

Q: Can the format string in a call of ...`printf` or ...`scanf` be a variable?

A: Sure; it can be any expression of type `char *`. This property makes the ...`printf` and ...`scanf` functions even more versatile than we've had reason to suspect. Consider the following classic example from Kernighan and Ritchie's *The C Programming Language*, which prints a program's command-line arguments, separated by spaces:

```
while (--argc > 0)
  printf((argc > 1) ? "%s " : "%s", *++argv);
```

The format string is the expression `(argc > 1) ? "%s " : "%s"`, which evaluates to `"%s "` for all command-line arguments but the last.

Q: Which library functions other than `clearerr` clear a stream's error and end-of-file indicators? [p. 565]

A: Calling `rewind` clears both indicators, as does opening or reopening the stream. Calling `ungetc`, `fseek`, or `fsetpos` clears just the end-of-file indicator.

Q: I can't get `feof` to work; it seems to return zero even at end-of-file. What am I doing wrong? [p. 565]

A: `feof` will only return a nonzero value when a previous read operation has failed; you can't use `feof` to check for end-of-file *before* attempting to read. Instead, you should first attempt to read, then check the return value from the input function. If the return value indicates that the operation was unsuccessful, you can then use `feof` to determine whether the failure was due to end-of-file. In other words, it's best not to think of calling `feof` as a way to *detect* end-of-file. Instead, think of it as a way to *confirm* that end-of-file was the reason for the failure of a read operation.

Q: I still don't understand why the I/O library provides macros named `putc` and `getc` in addition to functions named `fputc` and `fgetc`. According to Section 21.1, there are already two versions of `putc` and `getc` (a macro and a function). If we need a genuine function instead of a macro, we can expose the `putc` or `getc` function by undefining the macro. So why do `fputc` and `fgetc` exist? [p. 566]

A: Historical reasons. Prior to standardization, C had no rule that there be a true function to back up each parameterized macro in the library. `putc` and `getc` were traditionally implemented only as macros; `fputc` and `fgetc` were implemented only as functions.

***Q: What's wrong with storing the return value of `fgetc`, `getc`, or `getchar` in a `char` variable? I don't see how testing a `char` variable against `EOF` could give the wrong answer. [p. 568]**

A: There are two cases in which this test can give the wrong result. To make the following discussion concrete, I'll assume two's-complement arithmetic.

First, suppose that `char` is an unsigned type. (Recall that some compilers treat `char` as a signed type but others treat it as an unsigned type.) Now suppose that `getc` returns `EOF`, which we store in a `char` variable named `ch`. If `EOF` represents –1 (its typical value), `ch` will end up with the value 255. Comparing `ch` (an unsigned character) with `EOF` (a signed integer) requires converting `ch` to a signed integer (255, in this case). The comparison against `EOF` fails, since 255 is not equal to –1.

Now assume that `char` is a signed type instead. Consider what happens if `getc` reads a byte containing the value 255 from a binary stream. Storing 255 in the `ch` variable gives it the value –1, since `ch` is a signed character. Testing whether `ch` is equal to `EOF` will (erroneously) give a true result.

Q: The character input functions described in Section 22.4 require that the Enter key be pressed before they can read what the user has typed. How can I write a program that responds to individual keystrokes?

A: As you've noticed, the `getc`, `fgetc`, and `getchar` functions are buffered; they don't start to read input until the user has pressed the Enter key. In order to read characters as they're entered—which is important for some kinds of programs—you'll need to use a nonstandard library that's tailored to your operating system. In UNIX, for example, the `curses` library often provides this capability.

Q: When I'm reading user input, how can I skip all characters left on the current input line?

A: One possibility is to write a small function that reads and ignores all characters up to (and including) the first new-line character:

```
void skip_line(void)
{
  while (getchar() != '\n')
    ;
}
```

Another possibility is to ask `scanf` to skip all characters up to the first new-line character:

```
scanf("%*[^\n]");    /* skips characters up to new-line */
```

`scanf` will read all characters up to the first new-line character, but not store them anywhere (the `*` indicates assignment suppression). The only problem with using `scanf` is that it leaves the new-line character unread, so you may have to discard it separately.

Whatever you do, don't call the `fflush` function:

```
fflush(stdin);    /* effect is undefined */
```

Although some implementations allow the use of `fflush` to "flush" unread input, it's not a good idea to assume that all do. `fflush` is designed to flush *output* streams; the C standard states that its effect on input streams is undefined.

Q: Why is it not a good idea to use `fread` and `fwrite` with text streams? [p. 571]

A: One difficulty is that, under some operating systems, the new-line character becomes a pair of characters when written to a text file (see Section 22.1 for details). We must take this expansion into account, or else we're likely to lose track of our data. For example, if we use `fwrite` to write blocks of 80 characters, some of the blocks may end up occupying more than 80 bytes in the file because of new-line characters that were expanded.

Q: Why are there two sets of file-positioning functions (`fseek`/`ftell` and `fsetpos`/`fgetpos`)? Wouldn't one set be enough? [p. 574]

A: `fseek` and `ftell` have been part of the C library for eons. They have one drawback, though: they assume that a file position will fit in a `long int` value. Since `long int` is typically a 32-bit type, this means that `fseek` and `ftell` may not work with files containing more than 2,147,483,647 bytes. In recognition of this problem, `fsetpos` and `fgetpos` were added to `<stdio.h>` when C89 was created. These functions aren't required to treat file positions as numbers, so they're not subject to the `long int` restriction. But don't assume that you have to use `fsetpos` and `fgetpos`; if your implementation supports a 64-bit `long int` type, `fseek` and `ftell` are fine even for very large files.

Q: Why doesn't this chapter discuss screen control: moving the cursor, changing the colors of characters on the screen, and so on?

A: C provides no standard functions for screen control. The C standard addresses only issues that can reasonably be standardized across a wide range of computers and operating systems; screen control is outside this realm. The customary way to solve this problem in UNIX is to use the `curses` library, which supports screen control in a terminal-independent manner.

Similarly, there are no standard functions for building programs with a graphical user interface. However, you can most likely use C function calls to access the windowing API (application programming interface) for your operating system.

Exercises

Section 22.1

1. Indicate whether each of the following files is more likely to contain text data or binary data:
   (a) A file of object code produced by a C compiler
   (b) A program listing produced by a C compiler
   (c) An email message sent from one computer to another
   (d) A file containing a graphics image

Section 22.2

2. Indicate which mode string is most likely to be passed to `fopen` in each of the following situations:
   (a) A database management system opens a file containing records to be updated.
   (b) A mail program opens a file of saved messages so that it can add additional messages to the end.
   (c) A graphics program opens a file containing a picture to be displayed on the screen.
   (d) An operating system command interpreter opens a "shell script" (or "batch file") containing commands to be executed.

3. Find the error in the following program fragment and show how to fix it.

```
FILE *fp;

if (fp = fopen(filename, "r")) {
  read characters until end-of-file
}
fclose(fp);
```

Section 22.3

4. Show how each of the following numbers will look if displayed by `printf` with `%#012.5g` as the conversion specification:
   (a) 83.7361
   (b) 29748.6607
   (c) 1054932234.0
   (d) 0.0000235218

5. Is there any difference between the `printf` conversion specifications `%.4d` and `%04d`? If so, explain what it is.

W *6. Write a call of `printf` that prints

`1 widget`

if the `widget` variable (of type `int`) has the value 1, and

n `widgets`

otherwise, where *n* is the value of `widget`. You are not allowed to use the `if` statement or any other statement; the answer must be a single call of `printf`.

*7. Suppose that we call `scanf` as follows:

```
n = scanf("%d%f%d", &i, &x, &j);
```

(`i`, `j`, and `n` are `int` variables and `x` is a `float` variable.) Assuming that the input stream contains the characters shown, give the values of `i`, `j`, `n`, and `x` after the call. In addition, indicate which characters were consumed by the call.

(a) `10•20•30¤`
(b) `1.0•2.0•3.0¤`
(c) `0.1•0.2•0.3¤`
(d) `.1•.2•.3¤`

W 8. In previous chapters, we've used the `scanf` format string `" %c"` when we wanted to skip white-space characters and read a nonblank character. Some programmers use `"%1s"` instead. Are the two techniques equivalent? If not, what are the differences?

Section 22.4

9. Which one of the following calls is *not* a valid way of reading one character from the standard input stream?

(a) `getch()`
(b) `getchar()`
(c) `getc(stdin)`
(d) `fgetc(stdin)`

W 10. The `fcopy.c` program has one minor flaw: it doesn't check for errors as it's writing to the destination file. Errors during writing are rare, but do occasionally occur (the disk might become full, for example). Show how to add the missing error check to the program, assuming that we want it to display a message and terminate immediately if an error occurs.

11. The following loop appears in the `fcopy.c` program:

```
while ((ch = getc(source_fp)) != EOF)
  putc(ch, dest_fp);
```

Suppose that we neglected to put parentheses around `ch = getc(source_fp)`:

```
while (ch = getc(source_fp) != EOF)
  putc(ch, dest_fp);
```

Would the program compile without an error? If so, what would the program do when it's run?

12. Find the error in the following function and show how to fix it.

```
int count_periods(const char *filename)
{
  FILE *fp;
  int n = 0;
```

```
    if ((fp = fopen(filename, "r")) != NULL) {
      while (fgetc(fp) != EOF)
        if (fgetc(fp) == '.')
          n++;
      fclose(fp);
    }

    return n;
}
```

13. Write the following function:

```
int line_length(const char *filename, int n);
```

The function should return the length of line `n` in the text file whose name is `filename` (assuming that the first line in the file is line 1). If the line doesn't exist, the function should return 0.

Section 22.5

W 14. (a) Write your own version of the `fgets` function. Make it behave as much like the real `fgets` function as possible; in particular, make sure that it has the proper return value. To avoid conflicts with the standard library, don't name your function `fgets`.

(b) Write your own version of `fputs`, following the same rules as in part (a).

Section 22.7

W 15. Write calls of `fseek` that perform the following file-positioning operations on a binary file whose data is arranged in 64-byte "records." Use `fp` as the file pointer in each case.

(a) Move to the beginning of record `n`. (Assume that the first record in the file is record 0.)
(b) Move to the beginning of the last record in the file.
(c) Move forward one record.
(d) Move backward two records.

Section 22.8

16. Assume that `str` is a string that contains a "sales rank" immediately preceded by the # symbol (other characters may precede the # and/or follow the sales rank). A sales rank is a series of decimal digits possibly containing commas, such as the following examples:

```
989
24,675
1,162,620
```

Write a call of `sscanf` that extracts the sales rank (but not the # symbol) and stores it in a string variable named `sales_rank`.

Programming Projects

1. Extend the `canopen.c` program of Section 22.2 so that the user may put any number of file names on the command line:

```
canopen foo bar baz
```

The program should print a separate `can be opened` or `can't be opened` message for each file. Have the program terminate with status `EXIT_FAILURE` if one or more of the files can't be opened.

W 2. Write a program that converts all letters in a file to upper case. (Characters other than letters shouldn't be changed.) The program should obtain the file name from the command line and write its output to `stdout`.

3. Write a program named `fcat` that "concatenates" any number of files by writing them to standard output, one after the other, with no break between files. For example, the following command will display the files `f1.c`, `f2.c`, and `f3.c` on the screen:

   ```
   fcat f1.c f2.c f3.c
   ```

   `fcat` should issue an error message if any file can't be opened. *Hint:* Since it has no more than one file open at a time, `fcat` needs only a single file pointer variable. Once it's finished with a file, `fcat` can use the same variable when it opens the next file.

W 4. (a) Write a program that counts the number of characters in a text file.

(b) Write a program that counts the number of words in a text file. (A "word" is any sequence of non-white-space characters.)

(c) Write a program that counts the number of lines in a text file.

Have each program obtain the file name from the command line.

5. The `xor.c` program of Section 20.1 refuses to encrypt bytes that—in original or encrypted form—are control characters. We can now remove this restriction. Modify the program so that the names of the input and output files are command-line arguments. Open both files in binary mode, and remove the test that checks whether the original and encrypted characters are printing characters.

W 6. Write a program that displays the contents of a file as bytes and as characters. Have the user specify the file name on the command line. Here's what the output will look like when the program is used to display the `pun.c` file of Section 2.1:

```
Offset          Bytes                    Characters
------  -----------------------------    ----------
     0  23 69 6E 63 6C 75 64 65 20 3C    #include <
    10  73 74 64 69 6F 2E 68 3E 0D 0A    stdio.h>..
    20  0D 0A 69 6E 74 20 6D 61 69 6E    ..int main
    30  28 76 6F 69 64 29 0D 0A 7B 0D    (void)..{.
    40  0A 20 20 70 72 69 6E 74 66 28    .  printf(
    50  22 54 6F 20 43 2C 20 6F 72 20    "To C, or
    60  6E 6F 74 20 74 6F 20 43 3A 20    not to C:
    70  74 68 61 74 20 69 73 20 74 68    that is th
    80  65 20 71 75 65 73 74 69 6F 6E    e question
    90  2E 5C 6E 22 29 3B 0D 0A 20 20    .\n");..
   100  72 65 74 75 72 6E 20 30 3B 0D    return 0;.
   110  0A 7D                            .}
```

Each line shows 10 bytes from the file, as hexadecimal numbers and as characters. The number in the `Offset` column indicates the position within the file of the first byte on the line. Only printing characters (as determined by the `isprint` function) are displayed; other characters are shown as periods. Note that the appearance of a text file may vary, depending on the character set and the operating system. The example above assumes that `pun.c` is a Windows file, so 0D and 0A bytes (the ASCII carriage-return and line-feed characters) appear at the end of each line. *Hint:* Be sure to open the file in `"rb"` mode.

7. Of the many techniques for compressing the contents of a file, one of the simplest and fastest is known as ***run-length encoding***. This technique compresses a file by replacing sequences of identical bytes by a pair of bytes: a repetition count followed by a byte to be repeated. For example, suppose that the file to be compressed begins with the following sequence of bytes (shown in hexadecimal):

   ```
   46 6F 6F 20 62 61 72 21 21 21 20 20 20 20 20
   ```

   The compressed file will contain the following bytes:

```
01 46 02 6F 01 20 01 62 01 61 01 72 03 21 05 20
```

Run-length encoding works well if the original file contains many long sequences of identical bytes. In the worst case (a file with no repeated bytes), run-length encoding can actually double the length of the file.

(a) Write a program named `compress_file` that uses run-length encoding to compress a file. To run `compress_file`, we'd use a command of the form

`compress_file` *original-file*

`compress_file` will write the compressed version of *original-file* to *original-file*`.rle`.

For example, the command

```
compress_file foo.txt
```

will cause `compress_file` to write a compressed version of `foo.txt` to a file named `foo.txt.rle`. *Hint:* The program described in Programming Project 6 could be useful for debugging.

(b) Write a program named `uncompress_file` that reverses the compression performed by the `compress_file` program. The `uncompress_file` command will have the form

`uncompress_file` *compressed-file*

compressed-file should have the extension `.rle`. For example, the command

```
uncompress_file foo.txt.rle
```

will cause `uncompress_file` to open the file `foo.txt.rle` and write an uncompressed version of its contents to `foo.txt`. `uncompress_file` should display an error message if its command-line argument doesn't end with the `.rle` extension.

8. Modify the `inventory.c` program of Section 16.3 by adding two new operations:

   - Save the database in a specified file.
   - Load the database from a specified file.

   Use the codes `d` (dump) and `r` (restore), respectively, to represent these operations. The interaction with the user should have the following appearance:

```
Enter operation code: d
Enter name of output file: inventory.dat

Enter operation code: r
Enter name of input file: inventory.dat
```

   Hint: Use `fwrite` to write the array containing the parts to a binary file. Use `fread` to restore the array by reading it from a file.

W 9. Write a program that merges two files containing part records stored by the `inventory.c` program (see Programming Project 8). Assume that the records in each file are sorted by part number, and that we want the resulting file to be sorted as well. If both files have a part with the same number, combine the quantities stored in the records. (As a consistency check, have the program compare the part names and print an error message if they don't match.) Have the program obtain the names of the input files and the merged file from the command line.

*10. Modify the `inventory2.c` program of Section 17.5 by adding the `d` (dump) and `r` (restore) operations described in Programming Project 8. Since the part structures aren't stored in an array, the `d` operation can't save them all by a single call of `fwrite`. Instead, it will need to visit each node in the linked list, writing the part number, part name, and quan-

tity on hand to a file. (Don't save the `next` pointer; it won't be valid once the program terminates.) As it reads parts from a file, the `r` operation will rebuild the list one node at a time.

11. Write a program that reads a date from the command line and displays it in the following form:

    ```
    September 13, 2010
    ```

    Allow the user to enter the date as either `9-13-2010` or `9/13/2010`; you may assume that there are no spaces in the date. Print an error message if the date doesn't have one of the specified forms. *Hint:* Use `sscanf` to extract the month, day, and year from the command-line argument.

12. Modify Programming Project 2 from Chapter 3 so that the program reads a series of items from a file and displays the data in columns. Each line of the file will have the following form:

    item, *price*, *mm*/*dd*/*yyyy*

    For example, suppose that the file contains the following lines:

    ```
    583,13.5,10/24/2005
    3912,599.99,7/27/2008
    ```

    The output of the program should have the following appearance:

    ```
    Item            Unit            Purchase
                    Price           Date
    583             $  13.50        10/24/2005
    3912            $ 599.99        7/27/2008
    ```

    Have the program obtain the file name from the command line.

13. Modify Programming Project 8 from Chapter 5 so that the program obtains departure and arrival times from a file named `flights.dat`. Each line of the file will contain a departure time followed by an arrival time, with one or more spaces separating the two. Times will be expressed using the 24-hour clock. For example, here's what `flights.dat` might look like if it contained the flight information listed in the original project:

    ```
    8:00 10:16
    9:43 11:52
    11:19 13:31
    12:47 15:00
    14:00 16:08
    15:45 17:55
    19:00 21:20
    21:45 23:58
    ```

14. Modify Programming Project 15 from Chapter 8 so that the program prompts the user to enter the name of a file containing the message to be encrypted:

    ```
    Enter name of file to be encrypted: message.txt
    Enter shift amount (1-25): 3
    ```

    The program then writes the encrypted message to a file with the same name but an added extension of `.enc`. In this example, the original file name is `message.txt`, so the encrypted message will be stored in a file named `message.txt.enc`. There's no limit on the size of the file to be encrypted or on the length of each line in the file.

15. Modify the `justify` program of Section 15.3 so that it reads from one text file and writes to another. Have the program obtain the names of both files from the command line.

16. Modify the `fcopy.c` program of Section 22.4 so that it uses `fread` and `fwrite` to copy the file in blocks of 512 bytes. (The last block may contain fewer than 512 bytes, of course.)

17. Write a program that reads a series of phone numbers from a file and displays them in a standard format. Each line of the file will contain a single phone number, but the numbers may be in a variety of formats. You may assume that each line contains 10 digits, possibly mixed with other characters (which should be ignored). For example, suppose that the file contains the following lines:

```
404.817.6900
(215) 686-1776
312-746-6000
877 275 5273
6173434200
```

The output of the program should have the following appearance:

```
(404) 817-6900
(215) 686-1776
(312) 746-6000
(877) 275-5273
(617) 343-4200
```

Have the program obtain the file name from the command line.

18. Write a program that reads integers from a text file whose name is given as a command-line argument. Each line of the file may contain any number of integers (including none) separated by one or more spaces. Have the program display the largest number in the file, the smallest number, and the median (the number closest to the middle if the integers were sorted). If the file contains an even number of integers, there will be two numbers in the middle; the program should display their average (rounded down). You may assume that the file contains no more than 10,000 integers. *Hint:* Store the integers in an array and then sort the array.

19. (a) Write a program that converts a Windows text file to a UNIX text file. (See Section 22.1 for a discussion of the differences between Windows and UNIX text files.)

(b) Write a program that converts a UNIX text file to a Windows text file.

In each case, have the program obtain the names of both files from the command line. *Hint:* Open the input file in `"rb"` mode and the output file in `"wb"` mode.